PayWise AI – Technical Requirements Document (TRD)

Version: 1.0 (one-day hackathon build) | **Companion to:** PRD v1.0 | **Environment:** PayPal Developer Sandbox only

1. Technical Summary

PayWise AI is a three-tier web app. A React + Vite single-page app talks over REST/JSON to a FastAPI service. FastAPI uses Supabase (Postgres, Auth, Realtime) for data, the PayPal Sandbox for payments and webhooks, and Gemini for explanations. A deterministic rule engine detects issues; Gemini only explains them.

Design principles

1. **Rules decide, AI explains.** Detection is deterministic, testable and fast.
2. **One source of truth for status.** A single state machine drives the UI, alerts and explanations.
3. **Demo reliability over completeness.** Every external dependency has a fallback.
4. **Contract first.** API shapes are fixed in hour 1 so four people can work in parallel.

2. System Architecture

Layer	Technology	Responsibility	Hosting
Frontend	React 18, Vite, React Router, TanStack Table	UI, forms, live validation, dashboard, realtime subscription	Vercel
API	Python 3.11, FastAPI, Pydantic v2, httpx	Auth check, pre-flight, payments, webhooks, explanations	Render (web service)
Data and Auth	Supabase (Postgres, Auth, Realtime, RLS)	Users, recipients, transactions, events, notifications	Supabase cloud
Payments	PayPal Sandbox (Payouts API, Orders v2, Webhooks)	Money movement and status events	PayPal

Layer	Technology	Responsibility	Hosting
AI	Google Gemini (Flash-class model, JSON output)	Plain-language explanation	Google
Tooling	GitHub, Postman	Version control, API testing	n/a

Request flow

1. Browser calls `POST /payments/preflight` → rule engine returns risk and checks.
2. Browser calls `POST /payments` → backend re-runs pre-flight, creates the PayPal payment, stores the transaction.
3. PayPal sends a webhook to `POST /webhooks/paypal` → signature verified → event stored → state machine updates the transaction → notification created.
4. Supabase Realtime pushes the row change to the browser, so the dashboard updates live.
5. Browser calls `POST /payments/{id}/explain` → rule output is sent to Gemini (or the template) → explanation cached on the transaction.

3. Repository and Folder Structure (revised)

```
paywise-ai/
├── frontend/
|   ├── src/
|   |   ├── components/        # UI parts, incl. components/dashboard/
|   |   ├── pages/             # Landing, Login, Register, Dashboard, Send,
History
|   |   ├── services/          # api.js (fetch wrapper), supabase.js
|   |   ├── hooks/             # useAuth, usePreflight, useTransactions
(realtime)
|   |   └── App.jsx
|   └── package.json
├── backend/
|   ├── main.py                # app, CORS, router registration, /health
|   ├── routes/                # auth.py, recipients.py, payments.py,
webhooks.py, notifications.py, simulator.py
|   ├── ai_engine/             # rules.py, issue_detector.py, explainer.py,
templates.py
|   ├── paypal/                # client.py, payouts.py, webhooks.py,
status_map.py
```

```
|   ├── schemas/                # Pydantic request/response models
|   ├── database/               # supabase client, queries
|   ├── tests/                  # test_rules.py, test_status_map.py
|   └── requirements.txt
├── docs/                       # PRD, TRD, workflow, api-contracts.md, demo-
scenarios.md
├── .env.example
├── .gitignore
└── README.md
```

Change from the original plan: `ai-engine/` and `paypal/` move inside `backend/` so Render deploys one service with clean Python imports.

4. Data Model (Supabase / Postgres)

All tables have Row Level Security enabled. Users can read and write only their own rows. The backend uses the service-role key only for webhook processing.

profiles

- `id` uuid PK (= auth.users.id), `full_name` text, `role` text (sender | recipient), `paypal_email` text, `paypal_verified` boolean, `sandbox_balance` numeric(12,2) default 500.00, `currency` text default 'USD', `created_at` timestamptz

recipients (saved contacts)

- `id` uuid PK, `owner_id` uuid FK profiles, `name` text, `email` text, `created_at` timestamptz. Unique on (`owner_id`, `email`).

transactions

- `id` uuid PK, `sender_id` uuid FK profiles, `recipient_email` text, `amount` numeric(12,2), `currency` text, `note` text
- `status` text (see state machine), `issue_code` text null, `risk_level` text
- `paypal_batch_id` text, `paypal_item_id` text, `sender_item_id` text unique
- `preflight` jsonb, `explanation` jsonb, `is_simulated` boolean default false
- `created_at`, `updated_at` timestamptz

payment_events (append-only audit log)

- `id` uuid PK, `transaction_id` uuid FK, `source` text (paypal | simulator | system), `event_type` text, `paypal_event_id` text **unique** (idempotency), `payload` jsonb, `created_at`

notifications

- `id` uuid PK, `user_id` uuid FK profiles, `transaction_id` uuid FK, `title` text, `body` text, `is_read` boolean default false, `created_at`

Enable Supabase Realtime on `transactions` and `notifications`.

5. Payment Status State Machine

Internal status	Meaning	Entered from	PayPal source
`created`	Record saved, not yet submitted	start	n/a
`pending`	Submitted, awaiting outcome	created	Payout item PENDING; capture pending
`on_hold`	Held for review or verification	pending	Payout item ONHOLD / HELD / BLOCKED
`unclaimed`	Recipient has no account or has not claimed	pending	Payout item UNCLAIMED
`completed`	Funds delivered	pending, on_hold, unclaimed	Payout item SUCCESS
`returned`	Unclaimed funds sent back to sender	unclaimed	Payout item RETURNED / REFUNDED
`failed`	Rejected or errored	created, pending	Payout item FAILED / DENIED

`paypal/status_map.py` holds the single mapping from PayPal event type to internal status. Invalid transitions are logged and ignored, so out-of-order webhooks cannot corrupt state.

6. Issue Detection Engine

Implemented in `backend/ai_engine/`. All functions are pure and unit-tested.

Issue code	Problem	Logic
`WRONG_RECIPIENT_EMAIL`	1	Regex format check; disposable-domain list; edit distance of 1-2 to a saved recipient or to common provider typos (gmial.com, gmal.com); first-time recipient flag
`NO_PAYPAL_ACCOUNT`	2	Email not in `profiles.paypal_email` → risk of *unclaimed*; confirmed later by UNCLAIMED event
`INCOMPLETE_VERIFICATION`	3	Matching profile has `paypal_verified = false`; confirmed later by hold/blocked events
`INSUFFICIENT_BALANCE`	4	`amount + estimated_fee > sandbox_balance`
`MANUAL_ACCEPTANCE`	5	Pending reason indicates recipient must accept; simulator-driven in demo
`ECHECK_DELAY`	6	Pending reason eCheck on bank-funded payment; simulator-driven in demo

Pre-flight response shape

```
{
  "risk_level": "low | medium | high",
  "can_proceed": true,
  "checks": [
    {"code": "WRONG_RECIPIENT_EMAIL", "status": "pass | warn | fail",
"message": "...", "suggestion": "..."}
  ]
}
```

Risk scoring: any `fail` → high; two or more `warn` → medium; one `warn` → medium; otherwise low. A `fail` on `INSUFFICIENT_BALANCE` or an invalid email sets `can_proceed = false`.

7. AI Explanation Service

- **Input:** `issue_code`, `status`, masked recipient (for example `s***@gmail.com`), amount, currency, role (sender or recipient). No PII beyond that.
- **Prompt contract:** system prompt requires plain English at roughly a 7th-grade level, uses hedged wording for inferred causes, and returns **JSON only** with keys `summary`, `cause`, `next_steps` (array of 2-4), `message_to_recipient`.
- **Model:** a fast Gemini Flash-class model with JSON response mode; temperature 0.3; 4-second timeout.
- **Fallback:** `templates.py` holds one hand-written explanation per issue code. Used on timeout, parse failure, or missing API key.
- **Caching:** the result is stored in `transactions.explanation` keyed by status and issue code; it is regenerated only when the status changes.

8. PayPal Integration

Strategy: use the real PayPal Sandbox wherever it is possible, and the Scenario Simulator where it is not. Both feed the **same** event handler, so the demo exercises the real code path.

Need	Approach
Send money to an email	Payouts API: `POST /v1/payments/payouts` with `recipient_type: EMAIL`, a unique `sender_item_id`, and amount in a single item
Unclaimed (problem 2)	Real: pay to a sandbox email that has no account → UNCLAIMED event
Held / blocked (problem 3)	Real if sandbox allows; otherwise simulated
Manual acceptance (5), eCheck (6)	Simulated using PayPal-shaped event payloads (these are pending reasons on Checkout-style captures)
Webhooks	Subscribe to payouts item events (SUCCEEDED, UNCLAIMED, HELD, BLOCKED, RETURNED, FAILED, DENIED, REFUNDED)

Need	Approach
Signature check	Call PayPal `POST /v1/notifications/verify-webhook-signature` with the headers and raw body; reject on failure

Webhook handler rules

1. Read the raw body; verify the signature; reject with 400 if invalid.
2. Insert into `payment_events` with `paypal_event_id` unique. A duplicate insert means a retry → return 200 and stop.
3. Map event to status via `status_map.py`; apply the state machine.
4. Create a notification for the sender; return 200 fast. Explanation generation happens lazily when the user opens the Explain panel.

OAuth: fetch and cache the client-credentials access token in memory; refresh before expiry.

Hour-1 verification (blocking): confirm Payouts is enabled on the sandbox business account and a webhook can reach the Render URL. If Payouts is not available, switch to Orders v2 (payee-based checkout) plus Simulator, and update `status_map.py` only.

9. API Specification

Base URL: `/api/v1`. All routes except `/health` and `/webhooks/paypal` require `Authorization: Bearer <Supabase JWT>`.

Method	Path	Purpose	Owner
GET	`/health`	Liveness (also used by uptime pinger)	M2
GET	`/me`	Current profile and balance	M2
GET, POST	`/recipients`	List and save recipients	M2
POST	`/payments/preflight`	Run checks, return risk summary	M3
POST	`/payments`	Re-run pre-flight, create PayPal payment, store transaction	M4

Method	Path	Purpose	Owner
GET	`/payments`	List the user's transactions (filter by status)	M2
GET	`/payments/{id}`	Transaction detail and event timeline	M2
POST	`/payments/{id}/explain`	Return cached or fresh explanation	M3
POST	`/webhooks/paypal`	PayPal webhook receiver	M4
POST	`/simulator/{scenario}`	Trigger a scenario (demo only, sandbox flag required)	M4
GET	`/notifications`	List alerts	M2
POST	`/notifications/{id}/read`	Mark as read	M2

Standard error shape: `{ "error": { "code": "string", "message": "human readable" } }` with correct HTTP status codes (400 validation, 401 auth, 404, 409 duplicate, 502 upstream failure).

Contract rule: FastAPI generates OpenAPI at `/docs`; `docs/api-contracts.md` mirrors the request and response shapes above. The frontend builds against mock responses until the real endpoint is merged.

10. Frontend Technical Design

Page	Key components	Data source
Landing	Hero, six-problems grid, CTA	static
Login / Register	Auth forms (Supabase Auth)	Supabase
Send	PaymentForm, RecipientCheck (debounced 400 ms), RiskSummary	`/payments/preflight`, `/payments`
Dashboard	BalanceCard, StatusCounts, RecentPayments, AlertsBell	`/me`, realtime
History	TransactionTable (TanStack Table), status filter	`/payments`

Page	Key components	Data source
Payment Detail	StatusTimeline, ExplainPanel, MessageToRecipient (copy button)	`/payments/{id}`, `/explain`
Simulator (demo)	Scenario buttons	`/simulator/{scenario}`

State and data: `useAuth` (session), `usePreflight` (debounced call), `useTransactions` (initial fetch plus Supabase Realtime subscription). Use plain `fetch` through one `api.js` wrapper that attaches the JWT and normalizes errors. Show loading, empty and error states on every data view.

UI rules: consistent status badge colors (completed green, pending amber, unclaimed orange, on hold purple, failed red); a persistent *Sandbox mode* banner; responsive down to tablet width.

11. Security Requirements

- RLS policies: `select/insert/update` limited to `auth.uid() = owner/sender/user id` on every user table; `payment_events` readable only through the owning transaction.
- FastAPI validates the Supabase JWT (JWKS or shared secret) on every protected route.
- Secrets (`PAYPAL_CLIENT_ID`, `PAYPAL_SECRET`, `PAYPAL_WEBHOOK_ID`, `GEMINI_API_KEY`, `SUPABASE_SERVICE_ROLE_KEY`) live only in Render/Vercel env vars and local `.env`; `.env` is git-ignored; `.env.example` lists names only.
- CORS allows only the Vercel origin and localhost.
- Input validation with Pydantic (email format, amount greater than 0 and under a cap, currency whitelist).
- Idempotency via unique `sender_item_id` (payments) and `paypal_event_id` (events).
- Mask recipient emails before sending to Gemini; never log secrets or full tokens.
- Simulator route is enabled only when `APP_ENV=sandbox`.

12. Environment Variables

Variable	Used by
`SUPABASE_URL`, `SUPABASE_ANON_KEY`	frontend and backend
`SUPABASE_SERVICE_ROLE_KEY`, `SUPABASE_JWT_SECRET`	backend only
`PAYPAL_CLIENT_ID`, `PAYPAL_SECRET`, `PAYPAL_WEBHOOK_ID`, `PAYPAL_BASE_URL` (sandbox)	backend only
`GEMINI_API_KEY`, `GEMINI_MODEL`	backend only
`FRONTEND_ORIGIN`, `APP_ENV`	backend
`VITE_API_BASE_URL`, `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`	frontend

13. Testing Strategy

- **Unit (pytest):** each rule function with good and bad inputs; `status_map` transitions including invalid and duplicate events.
- **API (Postman collection, shared in repo):** pre-flight cases, create payment, webhook with a valid and an invalid signature, simulator scenarios, auth failures.
- **Integration:** one real sandbox payment from UI to webhook to dashboard update.
- **Scenario matrix:** the six problems each have a seeded input and an expected pre-flight result, status and explanation (documented in `docs/demo-scenarios.md`).
- **Pre-demo smoke test (15 minutes):** login, pre-flight, payment, webhook, realtime update, explain, simulator, all against deployed URLs.

14. Deployment

- **Backend (Render):** web service, build `pip install -r requirements.txt`, start `uvicorn backend.main:app --host 0.0.0.0 --port $PORT`; env vars set in dashboard; `/health` as the health check; external pinger every 10 minutes to avoid sleep.
- **Frontend (Vercel):** root `frontend/`, build `npm run build`, output `dist`; SPA rewrite to `index.html`; env vars set in dashboard.

- **PayPal:** webhook URL set to `https://<render-app>/api/v1/webhooks/paypal`; copy the webhook ID into `PAYPAL_WEBHOOK_ID`.
- **Order:** deploy a hello-world backend and frontend **early** (by the end of the first third of the day) so integration is never blocked on deployment.

15. Technical Risks and Fallbacks

Risk	Fallback
Payouts not enabled in sandbox	Orders v2 plus Simulator; only `paypal/` and `status_map.py` change
Webhook signature verification fails in sandbox	Log and investigate first; as a last resort in sandbox only, gate with an env flag, and say so in the demo
Gemini slow, rate-limited, or malformed JSON	Template fallback with a 4 s timeout; cached explanations
Realtime subscription issues	Poll `/payments` every 5 seconds as a fallback
Render cold start	Pinger plus warm-up before demo
Merge conflicts in the final hours	Feature freeze at the agreed time; only bug-fix PRs after it

16. Definition of Done

A feature is done when it works **on the deployed environment**, handles loading and error states, has its scenario in `demo-scenarios.md` passing, and is merged to `main` through a reviewed PR.